PROJECT LEDGERLINE · DOCUMENT 1 OF 3

Technical Blueprint

Architecture, data design and build plan for v1 of a synthetic banking data platform

Version	1.0 · build baseline
Date	Monday 9 November 2026 · day 9 of the 90-day arc
Owner	Lake · CEO and systems architect
Builder	Claude Code, working from this blueprint
Status	Approved for the v1 build · target Thursday 10 December 2026 (day 40)
Companions	Document 2: Business Plan Blueprint · Document 3: CEO Memo

> **In one sentence:** Ledgerline is a synthetic bank built to prove three things: money moves correctly, data moves safely, and every number can be proven to agree.

Contents

1. How to use this blueprint . . . 2
2. Mission and scope . . . 2
3. Architecture . . . 3
4. Technology stack (locked) . . . 4
5. Repository layout . . . 4
6. Domain model (DDD-lite) . . . 5
7. BankCore database design . . . 5
8. Business processes . . . 8
9. End of day and the business date . . . 9
10. Privacy by design and security . . . 9
11. Data pipeline . . . 10
12. Reconciliation and controls . . . 12
13. Synthetic data generator and traps . . . 12
14. Testing strategy . . . 13
15. Cloud finish line: Microsoft Fabric . . . 13
16. Build plan and definition of done . . . 14
17. Working with Claude Code . . . 15
18. Decisions log . . . 16
19. v2 and v3 at a glance . . . 17
20. Risks and open questions . . . 17
21. Glossary . . . 18

1. How to use this blueprint

This is the single source of truth for building v1. Claude Code builds from it and Lake reviews against it. When the build and the blueprint disagree, one of them is changed on purpose and the change is recorded in the decisions log (section 18).

Sections	What they cover	Read them
2 to 5	Mission, architecture, stack, repository	First: the shape of the system
6 to 10	Domain model, database, processes, end of day, security	Before writing any BankCore code
11 to 14	Pipeline, reconciliation, synthetic data, testing	Before writing any data code
15 to 17	Cloud finish line, build plan, Claude Code brief	When planning a week or a ticket
18 to 20	Decisions, v2 and v3, risks	Before an interview, or before changing scope

Reference IDs used throughout. INV-xx are technical invariants (section 6). BR-xx are business rules from the Business Plan Blueprint. R1 to R5 are reconciliations (section 12). T1 to T10 are planted traps (section 13). ADR-xxx are decisions (section 18). LL-0xx are build tickets (section 17). Every business rule maps to a component and a test.

2. Mission and scope

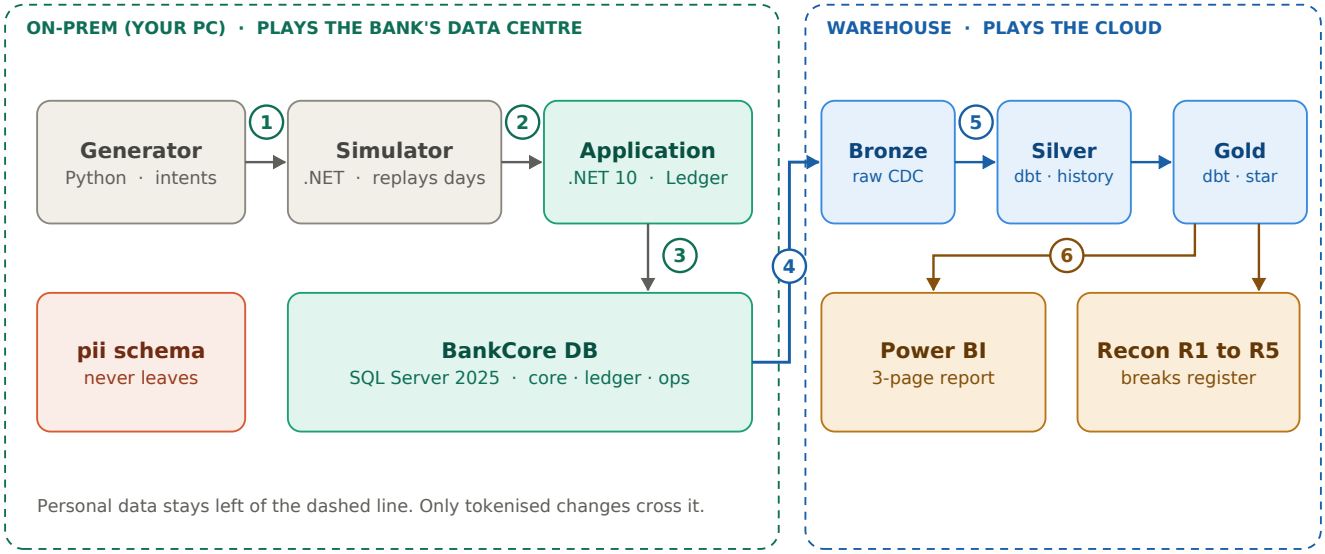
Pillar	What it means	How v1 proves it
Money moves correctly	Double-entry, all-or-nothing postings, no double charges, no overdrafts	Posting engine, idempotency keys, daily trial balance (R1)
Data moves safely	Only changes are shipped; personal data never leaves BankCore	Change data capture (CDC), personal-data split, token swap at extract
Everything is provable	Every number can be traced and reconciled	Control totals per layer, R1 to R5, breaks register, audit trail

What is in v1

In v1 (core)	Stretch, if time allows	Not in v1 (v2 and later)
• Salary (current) and Goal Savings accounts in AED • Salary credits from simulated WPS salary files • Own-account transfers and simplified payments out • Daily interest accrual, month-end capitalisation • End of day with a rolling business date • Thin admin: roles, maker-checker, product setup, batch control, audit • Synthetic data generator with ten planted traps • CDC pipeline to a local warehouse: bronze, silver, gold • Reconciliations R1 to R5 and a breaks register • Three-page Power BI report • Finish line: warehouse half rebuilt on Microsoft Fabric	• Debit card, card clearing and merchant data • Card spending page in the report	• Brokerage and USD investment accounts • Equity Research merge and market data vendor • Mortgages • Interbank payments and payment messaging • Fabric Mirroring comparison • Customer-facing app, streaming, multi-currency

3. Architecture

Two databases on one machine fake the bank's boundary. **BankCore** plays the bank's own data centre; **Warehouse** plays the cloud. At the finish line the warehouse half moves to Microsoft Fabric while BankCore stays put, which is exactly how many real banks split operations and analytics.



Step	What happens
1	The Python generator writes seeded, reproducible daily customer intents, including the planted traps.
2	The .NET simulator replays each day through the real application layer, then runs end of day.
3	Every money movement is posted through one stored procedure, ledger.usp_post_journal.
4	CDC changes leave BankCore with internal IDs swapped for tokens. The pii schema is never captured.
5	dbt builds silver (typed history) and gold (star schema) and runs data tests.
6	Power BI and the reconciliations read gold; anything that does not agree lands in the breaks register.

Components

Component	Role	Technology
Generator	Seeded daily intents and traps from the business plan assumptions	Python 3.12
Simulator and API	Replays intents day by day; minimal API for manual testing	.NET 10 console and minimal API
Application	Commands, domain rules, end-of-day jobs	.NET 10, EF Core 10
BankCore DB	System of record: customers, accounts, ledger, operations	SQL Server 2025 Developer
Extract	Reads CDC windows, swaps IDs for tokens, lands bronze, writes control totals	Python, pyodbc, SQL Server Agent
Warehouse DB	Bronze, silver, gold, reconciliation	SQL Server (local), then Fabric Warehouse
Transformations	Silver and gold models, tests, lineage	dbt-core with dbt-sqlserver, later dbt-fabric
Reporting	Bank overview, customers and cash flow, operations and controls	Power BI Desktop
Orchestration	End of day, extract, dbt, reconciliations	SQL Server Agent jobs

4. Technology stack (locked)

Layer	Choice	Why and how
Operational app	C# on .NET 10 (LTS) with EF Core 10	Bank-realistic; your domain-driven design experience carries over
Database	SQL Server 2025 Enterprise Developer edition	Free for development and test, full feature set including CDC and indexed views. SQL Server 2025 also adds a Standard Developer edition; either works for v1.
Schema	Numbered SQL migration scripts applied by DbUp	The database owns procedures, triggers, views and CDC. EF Core maps to the schema and never migrates it.
Generator and extract	Python 3.12, pyodbc, Microsoft ODBC Driver 18	The data engineering language
Transformations	dbt-core 1.9 with dbt-sqlserver 1.9	The SQL Server adapter is community-maintained and its versions track dbt-core, so pin both to the same minor version.
Reporting	Power BI Desktop	Free; reads gold and recon only
Tests	xUnit, pytest, dbt tests	Each layer tested in its own language
Orchestration	SQL Server Agent	CDC needs Agent running anyway; Fabric pipelines take over in the cloud half
Source control	Git and GitHub	Public repository with synthetic data only; doubles as the portfolio
Cloud finish line	Microsoft Fabric trial, on-premises data gateway, dbt-fabric	The trial capacity lasts 60 days. Fabric pipelines reach a local SQL Server through the gateway (version 3000.214.2 or later).

5. Repository layout

One repository, one solution, clear seams. The layout below is what LL-001 creates.

```
ledgerline/
├─ CLAUDE.md                     standing instructions for Claude Code (section 17)
├─ README.md                     what it is, how to run it, demo video link
├─ setup.ps1                     one command: create databases, migrate, seed, test
├─ docs/
│  ├─ blueprint/                 these three documents
│  ├─ decisions/                 ADR-001.md ... one file per decision
│  ├─ traps.md                   expected outcome of every planted trap
│  └─ parking-lot.md             ideas that are NOT in v1
├─ db/
│  ├─ bankcore/migrations/       0001_schemas.sql, 0002_core.sql, 0003_ledger.sql ...
│  └─ warehouse/migrations/      bronze, etl and recon schemas
├─ src/
│  ├─ Ledgerline.Domain/         value objects, aggregates, invariants
│  ├─ Ledgerline.Application/    commands, end-of-day jobs, ports
│  ├─ Ledgerline.Infrastructure/ EF Core, posting-engine adapter, clock, calendar
│  ├─ Ledgerline.Api/            minimal API for manual testing
│  └─ Ledgerline.Simulator/      replays generated intents one business day at a time
├─ tests/
│  ├─ Ledgerline.Domain.Tests/   xUnit: invariants in code
│  └─ Ledgerline.Db.Tests/       xUnit against SQL Server: constraints, triggers, posting proc
├─ generator/                    Python: config.yaml, seeded intents, traps
├─ pipeline/                     Python: CDC extract, token swap, control totals
├─ dbt/                          sources, staging, silver, gold, tests
├─ powerbi/                      report file and measure definitions
└─ ops/                          SQL Server Agent job scripts, replay.ps1, runbook.md
```

6. Domain model (DDD-lite)

One bounded context: **Ledger**. The model stays deliberately small. Every rule that protects money is enforced twice: once in the domain, once in the database.

Building block	Type	Responsibility
Money	Value object	Amount (decimal) plus ISO 4217 currency. Rejects floating point, mixed currencies and more than four decimal places.
BusinessDate	Value object	The bank's working day (Asia/Dubai, UTC+4, no daylight saving). Rolls only at end of day.
Account	Aggregate	Product, currency and status (open, frozen, closed). Decides whether a debit is allowed.
Journal	Aggregate	A balanced set of entries with one idempotency key. Cannot be constructed unbalanced.
TransferRequest	Aggregate	A customer intent with a client-generated request ID. A replay returns the original result.
PendingChange	Aggregate	A maker-checker request (rate, fee, reversal, manual adjustment). Approved only by a different user.
Product, RateSchedule	Entities	Effective-dated rates and fees, defined as data and never hard-coded.

Invariants

ID	Invariant	Enforced in the domain	Enforced in the database
INV-01	Every journal balances: entries sum to zero per currency	Journal cannot be built unbalanced	Posting procedure check plus AFTER INSERT trigger
INV-02	Ledger rows are never updated or deleted	No update or delete methods exist	DENY UPDATE, DELETE plus INSTEAD OF triggers
INV-03	A repeated idempotency key has no new effect	Handler returns the stored result	UNIQUE journal.idempotency_key; request_id primary key
INV-04	No customer account goes below zero	Account.CanDebit(amount)	Balance check under UPDLOCK in the posting procedure
INV-05	Frozen or closed accounts cannot be debited	Account status rules	Status check in the posting procedure
INV-06	Interest accrues at most once per account per day	Accrual is a pure function of (account, date)	Primary key (account_id, accrual_date)
INV-07	Makers cannot approve their own changes	PendingChange.Approve(checker)	CHECK (checker_user_id <> maker_user_id)
INV-08	Postings use the open business date only	BusinessDate comes from the calendar	Procedure reads the single OPEN date
INV-09	Personal data lives only in the pii schema	Identity behind a dedicated port	Schema permissions; CDC never enabled on pii

7. BankCore database design

7.1 Schemas and tables

Four schemas: **core** (customers without personal data, products, accounts, cards), **ledger** (postable accounts, journals, entries, accruals), **ops** (calendar, users, approvals, audit, batch runs, transfer requests) and **pii** (identities and the token vault).

Table	Purpose	Key columns and constraints
core.customer	Customer profile without personal data	customer_id PK; persona_code; home_emirate; kyc_status; onboarded_on; status

Table	Purpose	Key columns and constraints
pii.customer_identity	Personal data; never replicated	customer_id PK/FK; full_name; date_of_birth; national_id_synthetic; phone; email; address
pii.token_vault	Maps internal IDs to warehouse tokens	(entity_type, entity_id) PK; token UNIQUEIDENTIFIER UNIQUE DEFAULT NEWID()
core.product	Product definitions	product_code PK; product_type; currency_code; interest_bearing
core.product_rate	Effective-dated interest rates	(product_code, effective_from) PK; annual_rate DECIMAL(9,6); day_count 'ACT/365'; approved_change_id FK
core.fee_schedule	Effective-dated fees	(fee_code, effective_from) PK; amount DECIMAL(19,4); approved_change_id FK
core.account	Customer accounts	account_id PK; customer_id FK; product_code FK; iban CHAR(23) UNIQUE (synthetic, valid mod-97); currency_code; status CHECK IN ('OPEN','FROZEN','CLOSED'); opened_on; row_version
core.beneficiary	Payees for payments out	beneficiary_id PK; customer_id FK; category CHECK IN ('RENT','UTILITIES','TELECOM','REMITTANCE','SCHOOL','OTHER')
core.payment_out	Simplified outbound payments	payment_id PK; account_id FK; beneficiary_id FK; amount; category; journal_id FK UNIQUE
core.card (stretch)	Debit cards	card_id PK; account_id FK; card_token UNIQUE; pan_last4; status; expiry_ym. No full card number is ever stored.
core.card_transaction (stretch)	Card purchases	card_txn_id PK; card_id FK; merchant_descriptor (raw text); mcc CHAR(4); amount; journal_id FK UNIQUE
ledger.ledger_account	Every postable account	ledger_account_id PK; kind CHECK IN ('CUSTOMER','GL'); account_id UNIQUE NULL; gl_code UNIQUE NULL; CHECK exactly one of account_id or gl_code
ledger.journal	Header of each balanced posting	journal_id PK; journal_type; business_date; posted_at_utc; idempotency_key UNIQUE NOT NULL; reverses_journal_id UNIQUE NULL
ledger.entry	The posting lines	entry_id PK; journal_id FK; ledger_account_id FK; business_date; amount DECIMAL(19,4) CHECK (amount <> 0); currency_code. Positive = debit, negative = credit.
ledger.interest_accrual	Daily accrual memo	(account_id, accrual_date) PK; balance_basis; annual_rate; accrued DECIMAL(19,6); capitalised_journal_id FK NULL
ops.business_calendar	The bank's working day	business_date PK; status CHECK IN ('OPEN','EOD_RUNNING','CLOSED'); filtered unique index allows one OPEN row
ops.transfer_request	Idempotent customer intents	request_id PK (client-generated); from_account_id; to_account_id; amount; status; journal_id; response_json
ops.pending_change	Maker-checker queue	change_id PK; change_type; payload_json; maker_user_id; checker_user_id; status; CHECK (checker_user_id <> maker_user_id)
ops.app_user	Staff users and roles	user_id PK; username UNIQUE; role_code CHECK IN ('CUSTOMER_SERVICE','OPERATIONS','RISK','ADMIN','SYSTEM')
ops.audit_event	Append-only audit trail	event_id PK; occurred_at_utc; actor_user_id; event_type; entity_type; entity_id; before_json; after_json
ops.batch_run	Log of every job step	run_id PK; job_name; business_date; attempt; status; rows_processed; started_at_utc; finished_at_utc; error

7.2 Chart of accounts (the bank's own ledger accounts)

GL code	Name	Type	Used by
1010	Cash at central bank	Asset	Salary inflows, clearing settlements, initial capital
1250	Outbound payments clearing	Clearing	Payments out; settled to 1010 at end of day (nets to zero, R5)
1300	Card settlement clearing (stretch)	Clearing	Card purchases; settled to 1010 at end of day (R5)
3000	Share capital	Equity	Day-one capital: Dr 1010, Cr 3000

GL code	Name	Type	Used by
4100	Fee income	Income	Scheduled fees
5100	Interest expense	Expense	Month-end interest capitalisation

Customer accounts are liabilities of the bank: a credit increases what the bank owes the customer. The displayed customer balance is therefore the negative of the sum of entries.

7.3 The posting engine

Exactly one door into the ledger. The application calls a stored procedure with a table-valued parameter of lines; nothing else can write ledger rows.

```
CREATE TYPE ledger.journal_line AS TABLE (
    ledger_account_id BIGINT        NOT NULL,
    amount            DECIMAL(19,4) NOT NULL,     -- + debit, - credit
    currency_code     CHAR(3)       NOT NULL );

EXEC ledger.usp_post_journal
     @idempotency_key = N'TRF-6f1c2a90-...',      -- client request ID
     @journal_type    = 'TRANSFER',
     @lines           = @lines,                   -- inserted in ONE statement
     @journal_id      = @journal_id OUTPUT;
```

1. If the idempotency key already exists, return the existing journal_id and stop. Nothing else happens.
2. Validate: at least two lines, no zero amounts, every currency nets to zero, every account exists and is postable.
3. Lock the debited customer accounts with UPDLOCK in ascending account_id order (consistent order prevents deadlocks); reject if frozen or closed (INV-05) or if the debit would take the balance below zero (INV-04).
4. Insert the journal and all of its entries in one transaction, stamping each entry with the open business date.
5. Write an ops.audit_event row and commit.

> **Defence in depth.** The application role may EXECUTE the procedure but has no INSERT, UPDATE or DELETE rights on ledger tables. A backstop AFTER INSERT trigger re-checks that every affected journal nets to zero, and INSTEAD OF UPDATE and DELETE triggers keep the ledger append-only even for an administrator having a bad day.

7.4 Balances: derived, never stored

A stored balance column can drift away from the ledger. Ledgerline has none. Balances come from an indexed view that SQL Server keeps up to date inside the same transaction as each posting.

```
CREATE VIEW ledger.v_account_balance WITH SCHEMABINDING AS
SELECT e.ledger_account_id,
       SUM(e.amount) AS net_amount,              -- customer balance = -net_amount
       COUNT_BIG(*)  AS entry_count
FROM   ledger.entry AS e
GROUP  BY e.ledger_account_id;

CREATE UNIQUE CLUSTERED INDEX ux_v_account_balance
    ON ledger.v_account_balance (ledger_account_id);
```

- Read it WITH (NOEXPAND) so the stored index is always used.
- Indexed views need specific SET options (ANSI_NULLS, QUOTED_IDENTIFIER and others ON); migration 0001 sets them.
- Point-in-time balances for interest and reconciliation use ledger.entry filtered on business_date, served by an index on (ledger_account_id, business_date) INCLUDE (amount).
- Trade-off (ADR-007): every posting updates one view row per account, which can become a hot spot at real-bank volumes. Synthetic volumes are far below that; the upgrade path is periodic balance

snapshots.

7.5 Money, time and rounding

- **Money** is DECIMAL(19,4) in SQL and decimal in C#. FLOAT, REAL and double are banned.
- **AED has two decimal places** (100 fils to the dirham). Customer postings are rounded to the fils; daily accruals are kept at six decimal places.
- **Rounding is half away from zero everywhere.** SQL Server's ROUND already behaves this way, but C# Math.Round defaults to banker's rounding (half to even), so always pass MidpointRounding.AwayFromZero. Two languages with two defaults is a classic source of one-fils breaks.
- **Timestamps** are datetime2(3) in UTC, in columns ending _utc. **Business dates** are DATE values in Asia/Dubai time (UTC+4, no daylight saving).
- **Clock injection:** code never calls DateTime.Now. It uses the injected TimeProvider and the open business date from ops.business_calendar. That is what makes day-by-day simulation possible.

7.6 Idempotency map

Operation	Idempotency key	Enforced by
Own-account transfer	request_id generated by the client at the moment of intent	ops.transfer_request PK; UNIQUE journal.idempotency_key
Payment out	PAY-{request_id}	UNIQUE journal.idempotency_key
Salary credit	SAL-{wps_file_id}-{line_no}	UNIQUE journal.idempotency_key
Card purchase (stretch)	CARD-{scheme_txn_id}	UNIQUE journal.idempotency_key
Daily interest accrual	Natural key (account_id, accrual_date)	Primary key on ledger.interest_accrual
Month-end capitalisation	INTCAP-{account_id}-{yyyy-mm}	UNIQUE journal.idempotency_key
Clearing settlement	SETTLE-{gl_code}-{business_date}	UNIQUE journal.idempotency_key
Fee	FEE-{fee_code}-{event_ref}	UNIQUE journal.idempotency_key
CDC extract to bronze	(__$start_lsn, __$seqval, __$operation)	UNIQUE index on each bronze table
Reconciliation break	(check_code, business_date, object_key)	UNIQUE index on recon.break

8. Business processes

Process	Trigger	Journal (Dr / Cr)	Rules
Open account	Customer service, KYC passed	None: creates core.account and its ledger account	BR-01, BR-02
Salary credit	WPS salary file line	Dr 1010 Cash at central bank / Cr salary account	BR-03; T9
Own-account transfer	Customer request with request_id	Dr source account / Cr destination account	BR-04; INV-03 to 05
Payment out (simplified)	Payment to a named beneficiary, with a category	Dr salary account / Cr 1250 Outbound clearing	BR-05
Card purchase (stretch)	Simulated card authorisation	Dr salary account / Cr 1300 Card clearing	BR-09
Clearing settlement	End of day	Dr 1250 (and 1300) / Cr 1010 Cash at central bank	R5
Interest accrual	End of day, every business date	Memo row in ledger.interest_accrual (no journal)	BR-06; INV-06

Process	Trigger	Journal (Dr / Cr)	Rules
Interest capitalisation	End of day on the last day of the month	Dr 5100 Interest expense / Cr savings account	BR-06; R4
Fee	Scheduled fee event	Dr customer account / Cr 4100 Fee income	BR-08
Reversal	Operations maker, Risk checker	Mirror image of the original journal	BR-10; INV-02
Rate or fee change	Admin or Operations maker, Risk checker	None: a new, future-dated schedule row	BR-11; INV-07

Simplification to know about: v1 records daily accruals as memo rows and posts interest once a month. A production core would post daily to an interest-payable account; the upgrade is one extra journal type and one GL account.

9. End of day and the business date

Real cores run on a business date, not the wall clock. Ledgerline is a digital bank, so it runs end of day (EOD) for every calendar day.

Step	What happens	Safe to re-run because
1. Cut-off	Business date D moves to EOD_RUNNING; new postings wait for D+1	Status transition is checked
2. Accrue interest	Each savings balance at end of D × rate effective on D ÷ 365	Primary key (account_id, accrual_date)
3. Capitalise (month end)	Sum the month's accruals plus the carried remainder; post the amount rounded to the fils; carry the new remainder	Key INTCAP-{account}-{yyyy-mm}
4. Settle clearing	Settle 1250 (and 1300) to cash at the central bank	Key SETTLE-{gl}-{D}
5. Controls	R1 trial balance and R5 clearing check; halt EOD and raise a HIGH break if either fails (BR-13)	Read-only checks
6. Close and roll	D becomes CLOSED, D+1 becomes OPEN	Filtered unique index allows one OPEN date
7. Log	One ops.batch_run row per step with row counts and timings	Append-only

SQL Server Agent schedules use the machine's local time zone, not Dubai time. Convert when you schedule the nightly run; the simulator triggers EOD itself after each simulated day.

10. Privacy by design and security

Personal data never leaves BankCore. Three layers make that structural, not a matter of discipline:

1. **Structural split.** Personal data lives only in pii.*. CDC is never enabled on those tables, so their changes cannot be shipped.
2. **Column-level capture.** Capture lists (the @captured_column_list option of sys.sp_cdc_enable_table) leave out identifying columns such as the IBAN.
3. **Token swap at extract.** The extract query replaces customer_id, account_id and card_id with random tokens from pii.token_vault before anything is written to bronze. The vault itself never leaves.

> Someone with full access to the warehouse still cannot name a customer. That is the point.

Database roles

Role	Can	Cannot
ll_app (.NET application)	EXECUTE posting and ops procedures; read core, ledger, ops; read pii for customer service screens	INSERT, UPDATE or DELETE any ledger table
ll_etl (extract job)	Read CDC change functions and the token vault	Read pii.customer_identity; write anything in BankCore
ll_dbt (warehouse)	Read bronze; write silver, gold and recon	Connect to BankCore
ll_report (Power BI)	Read gold and recon	Read bronze or silver

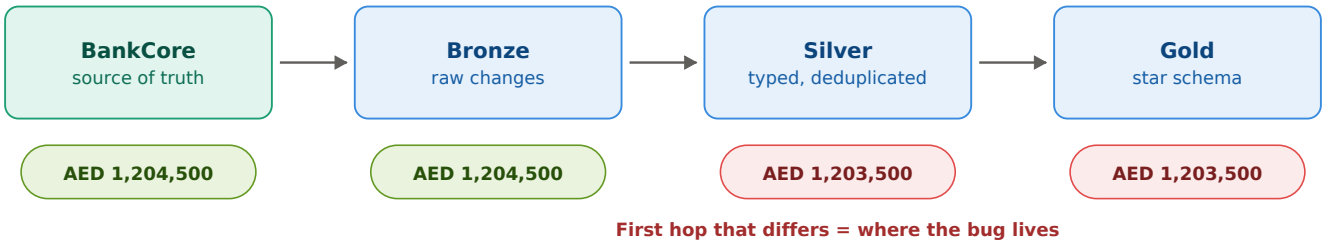
Human roles (customer service, operations, risk, admin) are enforced by the application on top of ll_app; who sees what is defined in the Business Plan Blueprint (BR-12).

Hygiene

- Windows authentication locally; no passwords in code. Connection strings live in .NET user-secrets and environment variables; .env files are git-ignored.
- Synthetic data only: no real names, ID numbers, IBANs or card numbers, ever. Synthetic IBANs use a fictional bank code with valid mod-97 check digits. Synthetic card numbers are Luhn-valid from a test prefix, and only a token and the last four digits are stored.
- Never log personal data. Structured logging uses a deny-list for pii fields.

11. Data pipeline

The pipeline is where the data engineering lives: capture changes, protect identities on the way out, land them raw, shape them twice, and count at every hop.



Reconciliation R2 stores row counts and amount sums at every layer on every run, so a break points straight at the hop that lost or gained data.

11.1 Change data capture

- Enable CDC on the database, then on core.customer, core.account, core.product_rate, core.beneficiary, core.payment_out, ledger.journal, ledger.entry, ledger.interest_accrual, ops.pending_change and ops.business_calendar (plus the card tables if built). Never on pii.*.
- SQL Server Agent must be running: CDC's capture and cleanup jobs are Agent jobs.
- The cleanup job keeps changes for three days by default (4,320 minutes). Raise it to 14 days in development so a missed week of extracts cannot silently lose changes.

11.2 Extract: idempotent by construction

1. Read the last processed LSN for each capture instance from Warehouse etl.cdc_watermark.
2. Read the current maximum LSN with sys.fn_cdc_get_max_lsn().
3. Pull every change in the window (last, max] with cdc.fn_cdc_get_all_changes_<instance>, joined to cdc.lsn_time_mapping for each change's commit time.
4. Swap internal IDs for tokens inside the same query, running as ll_etl in BankCore, so raw IDs never leave.
5. Insert into bronze, ignoring rows whose (LSN, seqval, operation) already exist.

6. Write control totals (row count, amount sum) to etl.load_audit, then advance the watermark **last**. If the job dies halfway, the next run repeats the window and the duplicates are simply ignored.

11.3 Bronze

One table per captured table (for example bronze.ledger_entry__cdc). Raw CDC columns are kept (__$start_lsn, __$seqval, __$operation, __$update_mask) plus commit_time_utc, _load_id and _extracted_at_utc. Bronze rows are never updated.

11.4 Silver (dbt)

- Staging models per bronze table: rename, cast, decode CDC operation codes.
- **History built from the change stream:** slowly changing tables (accounts, rates, customers) become type 2 history with valid_from and valid_to taken from each change's commit time, rather than from periodic snapshots.
- Append-only tables (journals, entries, accruals, payments) become incremental models keyed on their primary keys. Deduplication is by CDC keys, never by business values (trap T2).

11.5 Gold (dbt): star schema

Model	Grain	Feeds
dim_date	One row per calendar date, with month-end and payday-window flags	Everything
dim_customer	Customer token, persona, emirate, tenure band (current and history)	Customer pages
dim_account / dim_product	Account token, product, status history / product attributes	All facts
fct_ledger_entry	One row per ledger entry	Totals, reconciliation
fct_daily_balance	One row per account per business date, including quiet days	Balances, deposits, leftover at payday
fct_interest	Account by date accrual, plus month-end capitalisation	Interest expense, R4
fct_customer_cashflow_monthly	Customer by month: salary in, payments out by category, transfers to savings, leftover	Cash-flow page
fct_card_spend (stretch)	One row per card purchase with merchant category group	Card page
fct_ops_daily	Business date: EOD timings, breaks opened and closed, pipeline freshness	Operations page

> **fct_daily_balance is a balanced panel on purpose:** a row for every account on every business date, carrying the balance forward on quiet days. Chapter 2's lesson applies: a missing row is a pipeline bug, a zero is a fact.

11.6 Data tests

- Generic tests: unique, not_null, relationships and accepted_values on every model's keys and codes.
- Singular tests: every journal nets to zero in silver; no entry without a journal; the daily balance panel has no gaps; gold closing balances equal silver ledger sums; token columns contain GUIDs only, never raw IDs.

11.7 Orchestration

SQL Server Agent job	When	Steps
LL_EOD	After each business day (the simulator triggers it per simulated day)	Cut-off, accrue, capitalise, settle, R1 and R5, roll
LL_PIPELINE	Hourly, and immediately after LL_EOD	Extract, dbt build (silver, gold, tests), R2 to R4, breaks
LL_HOUSEKEEPING	Daily	CDC retention check, backups, log retention

Every job writes ops.batch_run or etl.load_audit rows, so failures surface on the Operations page of the report instead of in a log nobody reads.

12. Reconciliation and controls

Code	Check	Rule	On failure
R1	Trial balance	Every journal and the whole ledger net to zero per currency (BankCore)	EOD halts (BR-13); HIGH break
R2	Layer control totals	Row counts and amount sums match hop by hop: BankCore, bronze, silver, gold	Break at the first hop that differs
R3	Balance tie-out	Gold closing balance per account and date equals the BankCore point-in-time balance	Break per account
R4	Interest proof	Month's accruals plus opening remainder equal capitalised amount plus closing remainder	Break per account-month
R5	Clearing check	GL 1250 (and 1300) nets to zero after end-of-day settlement	EOD halts; HIGH break

Breaks register

recon.break holds: break_id, check_code, business_date, object_key (a token or a table name), expected, actual, difference, severity (HIGH, MEDIUM, LOW), owner, status (OPEN, INVESTIGATING, RESOLVED, ACCEPTED), opened_at_utc, resolved_at_utc, root_cause and notes. A unique key on (check_code, business_date, object_key) means re-running a check updates a break instead of duplicating it.

> **The missing-money drill.** When gold disagrees with BankCore, R2's stored totals show the first hop where the numbers diverge, so you know where to look before opening a single row.

13. Synthetic data generator and traps

13.1 Design: Python plans, .NET executes

- generator/ produces seeded, reproducible intent files, one JSON Lines file per business date: onboardings, salary file lines, transfers, payments out, rate changes, freezes (and card purchases if built).
- Ledgerline.Simulator replays each day through the real application layer, runs EOD and rolls the date. **No backdoor inserts:** every row in BankCore passed the same rules as a real request.
- v1 defaults: 2,000 customers, six months of history, seed 42. All parameters live in generator/config.yaml, taken from the Business Plan Blueprint (personas, salary bands, outflows, rates).
- The generator scales planned outflows to fit income, except where a trap deliberately says otherwise.

13.2 Planted traps

Trap	What the generator does	Expected outcome
T1 Duplicate request	Sends the same transfer request_id twice	One journal; the second call returns the original result
T2 Twin transfers	Two genuine AED 1,000 transfers, same accounts, same day	Both survive silver; R2 stays green
T3 Crash mid-accrual	Kills the accrual step halfway, then re-runs it	No double interest; R4 passes
T4 Late commit	A posting commits just after the extract reads the max LSN	Picked up by the next run; R2 differs once, then clears
T5 Frozen debit	Transfer out of a frozen account	Rejected (INV-05); audit event written
T6 Overdraft	Payment larger than the balance	Rejected (INV-04)
T7 Self-approval	A maker approves their own rate change	Blocked by the CHECK constraint (INV-07)
T8 Tamper	A direct UPDATE on ledger.entry	Denied by permissions and the immutability trigger
T9 Duplicate salary	The same WPS file line delivered twice	One credit only (key SAL-{file}-{line})
T10 Rounding edge	Accruals whose month total ends in a half fils	Same rounding in C# and SQL; R4 passes to the fils

14. Testing strategy

Layer	Tool	Proves	Examples
Domain	xUnit	Invariants hold in code	Unbalanced journal throws; overdraft rejected; self-approval rejected
Database	xUnit against a test database	The engine enforces rules even if the app is wrong	Direct INSERT into ledger.entry denied; duplicate key returns the same journal_id
Pipeline	pytest	Extract is idempotent and private	Re-running a window adds zero rows; no bronze column contains a raw ID
Models	dbt tests	Silver and gold are consistent	Journals net to zero; the panel has no gaps
System	Replay plus traps	The system catches what it should	T1 to T10 produce their expected outcomes
Break it on purpose	Demo script	You can explain every failure	Kill a job mid-run; delete a bronze row; change a rate unapproved

Definition of green: dotnet test, pytest and dbt build all pass, and the latest end of day shows zero open HIGH breaks.

15. Cloud finish line: Microsoft Fabric

Goal: move the warehouse half to the cloud while BankCore stays local, proving the hybrid boundary.

1. Start the Fabric trial only when v1 runs end to end locally. The trial lasts 60 days and provides a 64-capacity-unit trial capacity.
2. Create a workspace with a Fabric Warehouse, so T-SQL skills and the dbt models carry over.
3. Install the on-premises data gateway on the PC; Fabric pipelines need version 3000.214.2 or later.
4. Build a Fabric pipeline (or Copy job) that copies the tokenised bronze tables through the gateway. The token swap still happens locally, so personal data never crosses the boundary.
5. Point dbt at Fabric with the dbt-fabric adapter and run the same models. Record every dialect difference as an ADR.
6. Extend R2 across the boundary: local bronze, Fabric bronze, Fabric gold.

7. Point Power BI at Fabric. Write the before-and-after note for the README.

- **Guardrails:** everything lives in Git, so the trial ending loses nothing; no personal data in the cloud; the finish line must not stall v1 (hard stop on day 45).
- **Classic alternative:** Azure Data Factory with a self-hosted integration runtime does the same job in Azure; in Fabric, the on-premises data gateway plays that role.

16. Build plan and definition of done

Week	Dates (days)	Tickets	Exit criterion: must be demonstrable
W1 Foundations	Tue 10 – Sun 15 Nov days 10–15	LL-001 to LL-004	A balanced journal posts; an unbalanced one and a direct UPDATE are both refused
W2 The bank works	Mon 16 – Sun 22 Nov days 16–22	LL-005 to LL-009	Every BR and INV has a passing test; EOD rolls the business date; traps T1, T3, T5 to T7, T9 caught
W3 Data moves safely	Mon 23 – Sun 29 Nov days 23–29	LL-010 to LL-012	2,000-customer, six-month replay; trial balance zero every day; re-runnable extract; no raw IDs in the warehouse
W4 Everything provable	Mon 30 Nov – Sun 6 Dec days 30–36	LL-013 to LL-015 (+ LL-017 stretch)	Report live; R1 to R5 automated; all ten traps produce their expected outcome
Finish line	Mon 7 – Thu 10 Dec days 37–40	LL-016	Gold rebuilt in Fabric with matching control totals; README, demo video, v1.0 tag. Hard stop: Tue 15 Dec (day 45)

Definition of done for v1

- ☐ dotnet test, pytest and dbt build pass from a clean clone via setup.ps1
- ☐ The 2,000-customer, six-month replay completes and the trial balance is zero for every business date
- ☐ All ten traps produce their expected outcome, documented in docs/traps.md
- ☐ No personal data and no raw internal IDs exist anywhere in the warehouse (enforced by tests)
- ☐ R1 to R5 run automatically; the breaks register shows an owner and status for every break
- ☐ Power BI report: Bank overview, Customers and cash flow, Operations and controls
- ☐ Warehouse rebuilt in Fabric with matching control totals across the boundary
- ☐ README with architecture diagram, three-minute demo video, decisions log complete
- ☐ Lake can whiteboard the system and defend every ADR without notes

17. Working with Claude Code

17.1 CLAUDE.md starter (paste into the repository root)

```
# Ledgerline: standing instructions for Claude Code

Ledgerline is a fictional bank with synthetic data only, built as a financial data
engineering portfolio project. Read docs/blueprint before any task. Lake is the
architect: propose a plan, explain choices, and wait for review on design decisions.

## Non-negotiables
- Money is DECIMAL(19,4) in SQL and decimal in C#. Never float, real or double.
- Round half away from zero. In C# always pass MidpointRounding.AwayFromZero.
- Timestamps are UTC datetime2(3) in *_utc columns. Business dates are DATE (Asia/Dubai).
- Never call DateTime.Now. Use the injected TimeProvider and ops.business_calendar.
- The ledger is append-only. Corrections are reversal journals. No UPDATE or DELETE.
- Only ledger.usp_post_journal writes ledger rows. Every journal nets to zero.
- Every money-moving command carries an idempotency key backed by a UNIQUE constraint.
- Personal data lives only in the pii schema. Never log it, never replicate it.
- Synthetic data only: no real names, ID numbers, IBANs or card numbers.

## How we work
- One ticket = one branch = one small pull request. Stay inside the ticket.
- Write the failing test first for any rule in the blueprint (INV-xx, BR-xx).
- Explain non-obvious choices in the PR description. Record design decisions as
  docs/decisions/ADR-xxx.md.
- New ideas go in docs/parking-lot.md, never into the code.
- Schema changes are new numbered scripts in db/*/migrations. Never edit an applied one.

## Commands
- dotnet build; dotnet test
- python -m pytest generator pipeline
- cd dbt; dbt build
- ./ops/replay.ps1 -Customers 2000 -Months 6 -Seed 42
```

17.2 Ticket template

```
LL-0xx  <title>
Goal:            one sentence
Context:         blueprint sections; BR-xx and INV-xx references
Acceptance:      Given ... When ... Then ...  (three to six bullets)
Out of scope:    what not to touch
Tests required:  domain / database / pytest / dbt
```

17.3 The first seventeen tickets

ID	Ticket	Week
LL-001	Repository scaffold, solution structure, CLAUDE.md, setup.ps1	W1
LL-002	DbUp migration runner; schemas; SET options for indexed views	W1
LL-003	core and pii tables, token vault, constraints, synthetic IBAN generator with mod-97	W1
LL-004	Ledger tables, posting procedure with table-valued parameter, balance trigger, immutability, indexed view	W1
LL-005	Domain model (Money, BusinessDate, Account, Journal) with invariant tests	W2
LL-006	Business calendar, TimeProvider clock, EOD skeleton with batch_run logging	W2
LL-007	Open account, salary credit, idempotent transfers, payments out, minimal API	W2
LL-008	Interest accrual, month-end capitalisation, remainder carry, clearing settlement	W2
LL-009	Maker-checker, rate and fee schedules, audit events, application roles	W2
LL-010	Python generator (config.yaml, seeded intents, traps) and .NET simulator	W3
LL-011	Enable CDC with column lists; extract with LSN watermark; bronze; load audit	W3
LL-012	dbt project: sources, staging, type 2 history from CDC, silver tests	W3
LL-013	Gold star schema, daily balance panel, gold tests	W4

ID	Ticket	Week
LL-014	Reconciliations R1 to R5, breaks register, Agent jobs	W4
LL-015	Power BI report (three pages) and measure definitions	W4
LL-016	Fabric finish line: gateway, bronze copy, dbt-fabric, cross-boundary R2	Finish
LL-017	Stretch: debit card, card clearing, merchant data, card page	W4 if time

17.4 Review checklist (Lake, every pull request)

- ☐ Does it match the ticket, and nothing more?
- ☐ Can I explain every line? If not, ask Claude Code why before merging.
- ☐ Are the money, time and rounding rules followed?
- ☐ Is every new money path idempotent and covered by a test?
- ☐ Is there any personal data outside pii.*, or in a log?
- ☐ Did the new tests fail before the change? Ask to see red, then green.
- ☐ Is this a decision worth an ADR?

18. Decisions log

Each line becomes a one-page ADR in docs/decisions. Together they are the interview prep: every decision has a reason and a trade-off you chose with open eyes.

ADR	Decision	Why	Trade-off accepted
001	v1 is core banking plus the data pipeline	Data movement is the data engineering signal	Brokerage and admin screens wait
002	Local first; cloud is the v1 finish line	Zero cost, fast feedback, protects the 60-day trial	The hybrid boundary is proven late
003	Hand-built CDC pipeline before Fabric Mirroring	Depth of learning; strong interview story	More effort than the managed option
004	Split stack: .NET operational; SQL, Python and dbt for data	Mirrors real banks; shows both sides of the wall	Two toolchains
005	DDD-lite: one bounded context	Rules matter more than patterns	Less enterprise ceremony
006	The database enforces invariants too	Defence in depth	Some rules exist twice
007	Balances derived through an indexed view	No drift between balance and ledger	Hot rows at real scale; snapshots later
008	Personal-data split, CDC column lists, token swap at extract	Privacy by design; nothing to mask downstream	Warehouse analysis by token only
009	Idempotency keys on every money path and load	Retries and re-runs are always safe	Key design up front
010	Schema as numbered SQL scripts (DbUp); EF Core maps only	Procedures, triggers, views and CDC live in the database	No EF migrations convenience
011	dbt for silver and gold	Tests, lineage, documentation; in demand	One more tool
012	Type 2 history built from CDC commit times	True history instead of snapshot guesses	More modelling work
013	Business date with daily EOD; UTC storage	How real cores work; enables replay	Clock injection everywhere
014	Python generates intents; .NET replays them	No backdoor writes; reproducible runs	Two-step generation
015	Fabric Warehouse through the on-premises data gateway	Microsoft's direction; T-SQL carries over	Time-boxed trial; dialect differences

ADR	Decision	Why	Trade-off accepted
016	**Proposed:** fictional UAE digital bank in AED; v2 investing in USD at the fixed 3.6725 peg	Matches the UAE job target; the peg keeps FX simple	Replaces 'USD first' for banking. Confirm.
017	Equity Research merges in v2 as the market-data module; vendor choice deferred	One platform, one security master	Market data waits
018	Mortgage moves to v2	The heaviest product lands on a working platform	Domain strength shown later
019	**Proposed:** simplified payments out in v1	Without outflows balances only grow and cash-flow KPIs mean nothing	Slightly more v1 scope. Confirm.

19. v2 and v3 at a glance

Version	Window	Scope
v2	Days 41 to 70 (to Sat 9 Jan 2027)	Brokerage: USD investment account funded from AED at the peg, market orders, T+1 settlement, splits and dividends, bank-to-brokerage settlement reconciliation. Equity Research merge: security master refactor to internal IDs with effective-dated identifiers, free-tier vendor test, then one paid vendor. Mortgage book: amortisation, rate resets, arrears. Fabric Mirroring comparison on a cloned database so it never fights the hand-built CDC. Acquired-bank scenario: match a customer list with no shared ID (entity resolution, Chapter 4).
v3	Optional	Limit orders, multi-currency, streaming events, tax lots, a cash-flow credit score, and merchant matching from card descriptors (named entity recognition and resolution).

20. Risks and open questions

Risk	Guardrail
Scope creep	Parking lot. After day 10 nothing enters v1 unless something of equal size leaves.
Code Lake cannot explain	Review checklist and an explain-before-merge rule.
Hidden coupling to one machine	setup.ps1 from a clean clone is part of the definition of done.
The Fabric trial clock	Start it only at the finish line; keep everything in Git.
Rounding and time-zone bugs	Rules in CLAUDE.md; trap T10; reconciliation R4.
Trial blocked on the account	A tenant that already used a Fabric trial may be refused another; check early with a spare account so the finish line is not a surprise.
Tool versions	The stable dbt-sqlserver 1.9 line is tested up to SQL Server 2022 (newer adapter releases add 2025). If anything misbehaves, install SQL Server 2022 Developer instead; nothing else in this design changes.

Open questions

- Confirm ADR-016 (UAE framing in AED) and ADR-019 (payments out in v1).
- Debit card: stretch in v1, or first item of v2?
- Fabric Warehouse (proposed) or Lakehouse for the finish line?
- SQL Server installed natively on Windows (proposed) or in a container?

21. Glossary

Plain-language definitions of the terms this blueprint leans on. If you can explain each one in a sentence, you can explain the system.

Term	Meaning
Business date	The bank's working day. It moves forward only when end of day completes, whatever the wall clock says.
Change data capture (CDC)	A SQL Server feature that copies inserts, updates and deletes from the transaction log into change tables, so only changes need to be shipped.
LSN	Log sequence number: the position of a change in the transaction log. The extract uses it as a bookmark.
Watermark	The last LSN loaded successfully. It moves only after a load commits, which makes re-runs safe.
Idempotency	Doing the same operation twice has the same effect as doing it once.
Idempotency key	The unique ID that lets the system recognise a repeat: a client request ID, or a natural key such as account plus date.
Double-entry	Every posting has equal debits and credits, so the ledger always nets to zero.
Trial balance	The check that total debits equal total credits (R1).
Clearing account	A temporary bank account for money in transit. It should net to zero after settlement (R5).
Maker-checker	Four eyes: one person proposes a sensitive change, a different person approves it.
Medallion layers	Bronze holds raw data, silver holds cleaned and typed data, gold holds business-ready tables.
SCD type 2	History kept as extra rows with valid_from and valid_to, instead of overwriting old values.
Indexed view	A view whose results SQL Server stores and maintains automatically, like a table.
Tokenisation	Replacing an identifier with a random token. The token-to-identity map stays in the secure zone.
Control totals	Row counts and amount sums compared between systems or layers to prove nothing was lost or duplicated.
Balanced panel	One row per entity per date with no gaps, so a missing row signals a bug rather than a quiet day.

Sources

- Microsoft Learn, access on-premises data sources in Data Factory for Microsoft Fabric: https://learn.microsoft.com/en-us/fabric/data-factory/how-to-access-on-premises-data
- Microsoft Learn, Microsoft Fabric trial capacity: https://learn.microsoft.com/en-ca/fabric/fundamentals/fabric-trial
- Microsoft Learn, editions and components of SQL Server 2025: https://learn.microsoft.com/sql/sql-server/editions-and-components-of-sql-server-2025
- dbt docs, connect Microsoft SQL Server to dbt Core: https://docs.getdbt.com/reference/warehouse-setups/mssql-setup
- PyPI, dbt-sqlserver: https://pypi.org/project/dbt-sqlserver
- Microsoft Learn, mirroring SQL Server in Fabric: https://learn.microsoft.com/fabric/mirroring/sql-server